CISS350: Data Structures and Advanced Algorithms
Quiz q03

Name: eagraham1@cougars.ccis.edu Score:

For both questions, you are writing functions, not a complete programs.

TURN PAGE

Q1. Write a function that performs bubblesort using pointers and recursion that has the same behavior as our usual bubblesort (as specified in my 240 notes). Calling `bubblesort(start, end)` will perform bubblesort (in ascending order) on the values at address `start` up to `end - 1`. If `x[0..9]` is an array of size 10, then calling `bubblesort(&x[0], &x[10])` will sort `x[0..9]`. You cannot use for-, while-, do-while loops. You can only use recursion. A rough skeleton is given.

Answer:

```
void bubblesort_pass(int * start, int * end)
{
    if (start >= end)
    {
        return;
    }
    else
    {
      if(*start > *(start + 1))
      {
        int temp = *start;
        *start = *(start + 1);
        *(start + 1) = temp;
      }
        bubblesort_pass(start + 1, end);
    }
}

void bubblesort(int * start, int * end)
{
    if (start >= end)
    {
        return;
    }
    else
    {
        bubblesort_pass(start, end);
        bubblesort(start, end - 1);
    }
}
```

TURN PAGE

Q2. Write a function that performs binary search (using pointers and recursion) on the values at address `start` up to `end - 1`. The values are assumed to be in ascending order. If `x[0..9]` is an array of size 10, then calling `binarysearch(&x[0], &x[10], 42)` will return the *address* of 42 in `x[0..9]`. You cannot use for-, while-, do-while loops. You can only use recursion. A rough skeleton is given.

Answer:

```
int * binarysearch(int * start, int * end, int target)
{
    if (start > end)
    {
        return NULL; // target not found
    }
    else
    {
      int * mid = start + (end - start) / 2;
      if(*mid == target)
        return mid;
      else if (target < *mid)
        return binarysearch(start, mid - 1, target);
      else
        return binarysearch(mid + 1, end, target);

    }
}
```

Instructions

In the file `thispreamble.tex` look for

```
\renewcommand\AUTHOR{}
```

and enter your email address:

```
\renewcommand\AUTHOR{jdoe5@cougars.ccis.edu}
```

(This is not really necessary since alex will change that for you when you execute `make`.) In your bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

Enter your answers in `main.tex`. In the bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

For each question, you'll see boxes for you to fill. For small boxes, if you see

```
1 + 1 = \answerbox{}.
```

you do this:

```
1 + 1 = \answerbox{2}.
```

`answerbox` will also appear in "true/false" and "multiple-choice" questions.

For longer answers that need typewriter font, if you see

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
\end{answercode}
```

you do this:

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
int x;
\end{answercode}
```

`answercode` will appear in questions asking for code, algorithm, and program output. In this case, indentation and spacing is significant. For program output, I do look at spaces and newlines.

For long answers (not in typewriter font) if you see

```
What is the color of the sky?
\begin{answerlong}
\end{answerlong}
```

you can write

```
What is the color of the sky?
\begin{answerlong}
The color of the sky is blue.
\end{answerlong}
```

A question that begins with "T or F or M" requires you to identify whether it is true or false, or meaningless. "Meaningless" means something's wrong with the question and it is not well-defined. Something like "$1 + 2 = 4$" is either true or false (of course it's false). Something like "$1+_2 = 4$?" does not make sense.

When writing results of computations, make sure it's simplified. For instance write 2 instead of $1 + 1$.

Higher level classes.

For students beyond 245: You can put LaTeX commands in `answerlong`.

More examples of meaningless statements: Questions such as "Is $42 = 1+_2$ true or false?" or "Is $42 = \{2\}^{\{3\}}$ true or false?" does not make sense. "Is $P(42) = \{42\}$ true or false?" is meaningless because $P(X)$ is only defined if X is a set. For "Is $1 + 2 + 3$ true or false?", "$1 + 2 + 3$" is well–defined but as a "numerical expression", not as a "proposition", i.e., it cannot be true or false. Therefore "Is $1 + 2 + 3$ true or false?" is also not a well-defined question.

More examples of simplification: When you write down sets, if the answer is $\{1\}$, do not write $\{1, 1\}$. And when the values can be ordered, write the elements of the set in ascending order. When writing polynomials, begin with the highest degree term.

When writing a counterexample, always write the simplest.